



Data Structures and Algorithms in Python

Lecture 2: Pseudocode and Classes

Faculty of Data Science and Artificial Intelligence

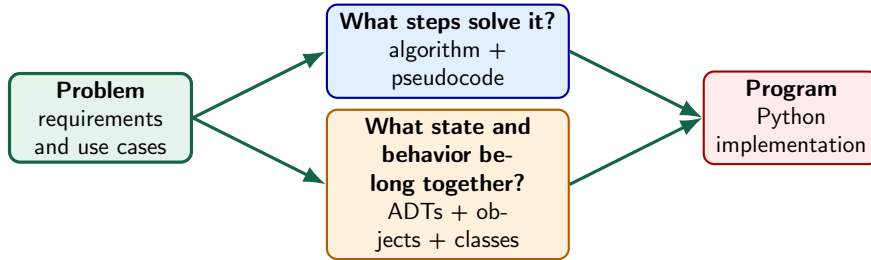
College of Technology – National Economics University

Learning outcomes

By the end of this lecture, students should be able to:

- ① Explain what pseudocode is and why it is useful before implementation.
- ② Write clear pseudocode using variables, parameters, control flow, loops, and function calls.
- ③ Explain why classes are useful for representing meaningful business and financial entities.
- ④ Distinguish a class from an object and explain attributes, methods, `self`, and encapsulation.
- ⑤ Explain how inheritance and abstract base classes can support ADT interfaces and multiple implementations.
- ⑥ Explain why classes provide a natural implementation mechanism for data structures.

Two complementary design questions



Pseudocode designs the process

It expresses the sequence, decisions, loops, and function calls needed to solve a problem without committing to Python syntax.

Classes design the abstraction

They organize related state, operations, and rules into objects with a clear public interface.

Algorithm and class design proceed in parallel and refine each other.

SECTION 1

Pseudocode

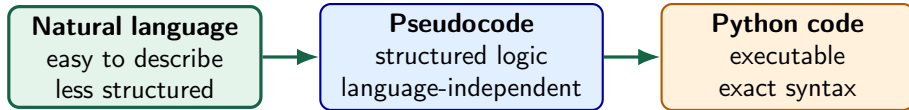
Pseudocode

- ➊ Definition
- ➋ When and why to use pseudocode
- ➌ Characteristics of good pseudocode
- ➍ A first example
- ➎ Guidelines for writing pseudocode
Variables, parameters, assignment, control flow, loops, and function calls.
- ➏ More examples
- ➐ From pseudocode to clean Python code

What is pseudocode?

Definition

Pseudocode is a structured, human-readable description of an algorithm, independent of the exact syntax of a programming language.



We may also use a [flowchart](#) to illustrate an algorithm before introducing its pseudocode.

Pseudocode – a simple example

Pseudocode

```
1 FUNCTION FIND_LARGEST
2 INPUT: a non-empty list of numbers
3 OUTPUT: the largest number
4
5     largest = the first number in numbers
6
7     FOR EACH number IN numbers DO
8         IF number > largest THEN
9             largest = number
10
11     RETURN largest
```

Python code

```
1 def find_largest(numbers):
2
3     largest = numbers[0]
4
5     for number in numbers[1:]:
6         if number > largest:
7             largest = number
8
9     return largest
```

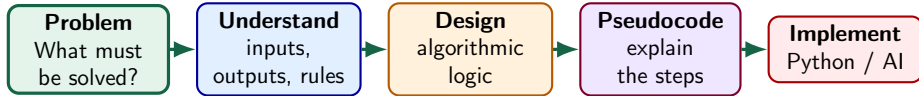
The control flow is the same; Python adds exact syntax.

Quiz: Trace a simple algorithm

An algorithm scans a list from left to right and keeps a variable `largest`. The input list is
[4, 9, 2, 11, 6].

- 1 What are the values of `largest` after seeing 9, 2, 11, and 6?
- 2 Why should this version of the algorithm require a non-empty list?

Why and when should we use pseudocode?



Why?

It helps

- think through problems systematically
- create implementation roadmaps
- serve as a universal language

When?

During

- planning phase
- team communication
- documentation

Main characteristics of pseudocode

- Uses plain English mixed with programming-like structure
- Focuses on logic rather than syntax
- Language-independent (can be converted to any programming language)
- Easy to read and understand by both programmers and non-programmers
- Uses common programming constructs like loops, conditionals, and variables

Quiz: Pseudocode fundamentals

Choose the best answer.

- ❶ What is the main purpose of pseudocode?
 - ❶ To replace testing
 - ❷ To describe algorithmic logic before implementation
 - ❸ To force a program to run faster
 - ❹ To avoid specifying inputs and outputs
- ❷ Which line is most appropriate in language-independent pseudocode?
 - ❶ `for i in range(len(a)):`
 - ❷ `FOR EACH item IN items DO`
 - ❸ `items.append(x)`
 - ❹ `print(x)`

How to write good pseudocode

Structure

- 1 State the inputs and expected output.
- 2 Give the algorithm or function a meaningful name.
- 3 Write one action per line.
- 4 Indent the bodies of decisions and loops.

Clarity

- 1 Use consistent keywords such as IF, FOR, and RETURN.
- 2 Choose meaningful variable names.
- 3 Include validation and boundary cases.
- 4 Avoid unnecessary Python-specific syntax.

A useful test

Another programmer should be able to implement the algorithm without asking what any step means.

Example 1: repetition with WHILE

Pseudocode

```
1 FUNCTION SUM_TO_N
2 INPUT: an integer n
3 OUTPUT: the sum from 1 to n
4
5 IF n < 1 THEN
6     report an error
7
8 total = 0
9 i = 1
10
11 WHILE i <= n DO
12     total = total + i
13     i = i + 1
14
15 RETURN total
```

Python function

```
1 def sum_to_n(n):
2     if n < 1:
3         raise ValueError("n >= 1 required")
4
5     total = 0
6     i = 1
7
8     while i <= n:
9         total += i
10        i += 1
11
12    return total
```

The loop repeats while the condition remains true; the counter must advance.

Example 2: solve a quadratic equation ($a \neq 0$)

Pseudocode

```
1 FUNCTION SOLVE_QUADRATIC
2 INPUT: real numbers a, b, c with  $a \neq 0$ 
3 OUTPUT: the real roots, or NONE
4
5     delta =  $b*b - 4*a*c$ 
6
7     IF delta > 0 THEN
8          $x1 = (-b + \text{SQRT}(\text{delta})) / (2*a)$ 
9          $x2 = (-b - \text{SQRT}(\text{delta})) / (2*a)$ 
10        RETURN x1, x2
11    ELSE IF delta = 0 THEN
12        RETURN  $-b / (2*a)$ 
13    ELSE
14        RETURN NONE
```

Python function

```
1 from math import sqrt
2
3 def solve_quadratic(a, b, c):
4     if a == 0:
5         raise ValueError("a must be non-zero")
6
7     delta = b * b - 4 * a * c
8
9     if delta > 0:
10        x1 = (-b + sqrt(delta)) / (2 * a)
11        x2 = (-b - sqrt(delta)) / (2 * a)
12        return x1, x2
13    if delta == 0:
14        return -b / (2 * a)
15    return None
```

The discriminant determines whether there are two, one, or no real roots.

Mini exercise: write the pseudocode

Problem statement: Shopping-cart total

Given the prices of n items in a shopping cart, calculate their sum. If the sum is at least 1,000,000 VND, apply a 10% discount. Otherwise, the customer pays the full sum.

Input

The first line contains an integer n ($1 \leq n \leq 10^5$). The second line contains n non-negative integer prices $p_1, p_2, \dots, p_n$.

Output

Print one number: the final amount the customer must pay, in VND.

Your task: 5 minutes

Write language-independent pseudocode. Clearly show:

- initialization,
- the loop,
- the discount decision,
- the returned result.

Introduction: Dijkstra's shortest-path algorithm

Goal: find the shortest distance from one source to every vertex in a weighted graph. **Assumption:** all edge weights are non-negative.

Pseudocode

```
1 FUNCTION DIJKSTRA
2 INPUT: weighted graph and source vertex
3 OUTPUT: distance and previous maps
4   FOR EACH vertex IN graph DO
5     distance[vertex] = INFINITY
6     previous[vertex] = NONE
7   distance[source] = 0
8   unvisited = SET of all vertices
9   WHILE unvisited is not empty DO
10    u = unvisited vertex with smallest distance
11    REMOVE u from unvisited
12    FOR EACH (v, weight) IN graph[u] DO
13      alternative = distance[u] + weight
14      IF alternative < distance[v] THEN
15        distance[v] = alternative
16        previous[v] = u
17  RETURN distance, previous
```

Python function

```
1 def dijkstra(graph, source):
2     distance = {v: float("inf") for v in graph}
3     previous = {v: None for v in graph}
4     distance[source] = 0
5     unvisited = set(graph)
6     while unvisited:
7         u = min(unvisited, key=distance.get)
8         unvisited.remove(u)
9         for v, weight in graph[u]:
10             alternative = distance[u] + weight
11             if alternative < distance[v]:
12                 distance[v] = alternative
13                 previous[v] = u
14     return distance, previous
```


From pseudocode to AI-generated code

- Large language models can help convert clear pseudocode into Python code.
- The pseudocode acts as a specification of the required logic, inputs, outputs, and rules.
- A more precise specification usually produces code that is easier to review and correct.
- Compare the generated code with the pseudocode and test normal and boundary cases.

Key point

AI can accelerate implementation, but the human remains responsible for problem solving, design decisions, and verification.

SECTION 2

Classes and Objects

Classes and Objects

- ① A first example of a class
- ② Why do we need classes?
- ③ Why classes are useful in Data Structures
- ④ More examples of classes
- ⑤ A class vs. an object: attributes, methods, and `self`
- ⑥ Guidelines for designing a class
- ⑦ From classes to data structures: implementing a Dynamic Array in Python

Objects in the real world

Dog

Attributes

- name, color
- age, weight

Methods

- bark()
- eat()
- run()

Car

Attributes

- brand, color
- speed, fuel level

Methods

- start()
- accelerate()
- brake()

Credit Card

Attributes

- customer, account
- balance, credit limit

Methods

- charge()
- make_payment()
- get_balance()

Objects have attributes and methods.

How should we represent many similar objects?

Suppose we want to represent two dogs in Python.

Dog 1: Milo

```
milo_name = "Milo"
milo_color = "Brown"
milo_age = 3
milo_weight = 12
def milo_bark():
    print("Woof!")
def milo_eat():
    print("Milo is eating")
def milo_run():
    print("Milo is running")
```

Dog 2: Max

```
max_name = "Max"
max_color = "Black"
max_age = 5
max_weight = 18
def max_bark():
    print("Woof!")
def max_eat():
    print("Max is eating")
def max_run():
    print("Max is running")
```

This works, but...

- Attributes of the same dog are stored in many separate variables.
- Similar functions are repeated for every dog.
- The code becomes difficult to manage as the number of dogs increases.

What if we have 100 dogs?

There must be a better way

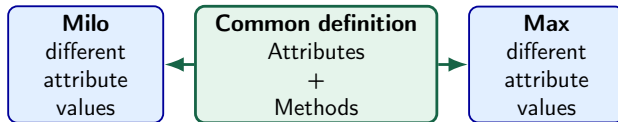
For every dog, we need the same structure:

Attributes

- name, color
- age, weight

Methods

- bark()
- eat()
- run()



What we need

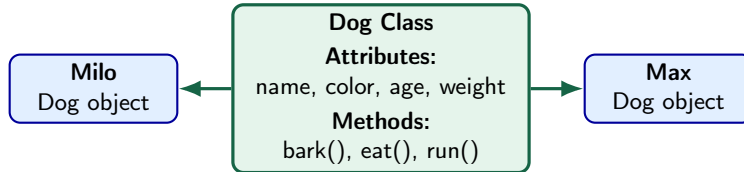
Define the common **attributes and methods once**, then create many individual dogs with different attribute values.

That common definition is a Class.

From objects to a class

The idea

We need a user-defined data type that specifies the **attributes and methods** shared by similar objects.



Class

A **class** defines a common set of **attributes and methods**.

Object

An **object** is an instance of a class with specific attribute values.

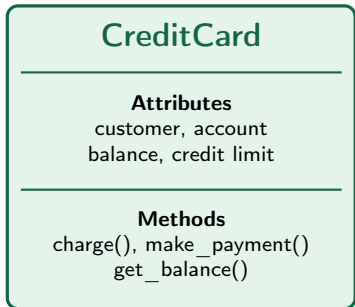
Quiz: Classes and objects

Classify each item as an **attribute**, a **method**, or an **object** in a BankAccount system.

- ① balance
- ② withdraw(amount)
- ③ alice_account
- ④ deposit(amount)
- ⑤ owner

A class defines the structure; an object is one concrete instance.

From a simple class to a business class



Same idea

Like a Dog or Car, a CreditCard has attributes and methods.

Objects

alice_card and bob_card belong to the same class but have different attribute values.

Rules

Methods must preserve rules such as

$$0 \leq \text{balance} \leq \text{credit limit}.$$

Guidelines for designing a class

1. What should it know?

Identify the object's **attributes**.

- account ID
- owner
- balance

2. What should it do?

Identify the object's **methods**.

- charge()
- make_payment()
- get_balance()

Attribute + Method $\Rightarrow$ Class

Pseudocode for class design

charge(amount)

```
METHOD CHARGE
INPUT: an amount
OUTPUT: TRUE if accepted; otherwise
        FALSE
    IF amount <= 0 THEN
        RETURN FALSE
    IF balance + amount > limit THEN
        RETURN FALSE
    balance = balance + amount
    RETURN TRUE
```

Uses: balance, limit

make_payment(amount)

```
METHOD MAKE_PAYMENT
INPUT: an amount
OUTPUT: TRUE if accepted; otherwise
        FALSE
    IF amount <= 0 THEN
        RETURN FALSE
    IF amount > balance THEN
        RETURN FALSE
    balance = balance - amount
    RETURN TRUE
```

Changes: balance

get_balance()

```
METHOD GET_BALANCE
INPUT: NONE
OUTPUT: the current balance
        RETURN balance
```

Reads state only

Key distinction

Query
reads attribute

Command
may change attribute

A method is an algorithm that operates on an object's state while preserving its rules.

A complete CreditCard class

```
1 class CreditCard:
2     def __init__(self, customer, account, limit):
3         self._customer = customer
4         self._account = account
5         self._limit = limit
6         self._balance = 0
7
8     def charge(self, price):
9         if price <= 0:
10             return False
11         if self._balance + price > self._limit:
12             return False
13         self._balance += price
14         return True
15
16     def make_payment(self, amount):
17         if amount <= 0 or amount > self._balance:
18             return False
19         self._balance -= amount
20         return True
21
22     def get_balance(self):
23         return self._balance
```

Attribute

Customer, account, limit, and balance belong to each object.

Method

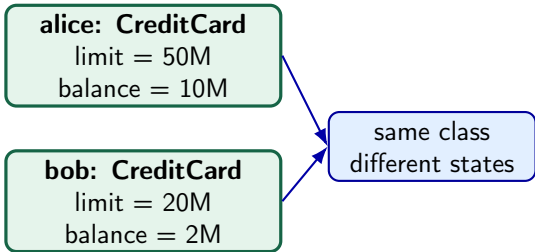
charge() and make_payment() change state only when their business rules are satisfied.

Encapsulation

Client code should use methods rather than modify _balance directly.

Class vs. object: one blueprint, many instances

```
1 alice = CreditCard(  
2     "Alice", "A001", 50_000_000)  
3  
4 bob = CreditCard(  
5     "Bob", "B002", 20_000_000)  
6  
7 alice.charge(10_000_000)  
8 bob.charge(2_000_000)
```



What is self?

self refers to the particular object on which a method is called.

Example 1: Bank Account

Problem

A bank needs to manage customer accounts. Each account has an owner, an account number, and a current balance.

Customers can deposit and withdraw money. However, a withdrawal should only be accepted if the amount is positive and the account has sufficient balance.

What should it know?

- account number
- owner
- balance

What should it do?

- deposit money
- withdraw money
- report balance

Problem $\Rightarrow$ Attribute + Method

Example 1: Designing the BankAccount class

Class design

```
CLASS BankAccount

    ATTRIBUTE
        account_id
        owner
        balance

    METHODS
        deposit(amount)
        withdraw(amount)
        get_balance()
```

Every method must preserve the rule
 $balance \geq 0$.

⇒ **The class combines data with the algorithms that operate on that data.**

Method pseudocode

```
DEPOSIT(amount)
    IF amount <= 0
        RETURN FALSE
    balance = balance + amount
    RETURN TRUE

WITHDRAW(amount)
    IF amount <= 0
        RETURN FALSE
    IF amount > balance
        RETURN FALSE
    balance = balance - amount
    RETURN TRUE

GET_BALANCE()
    RETURN balance
```

Example 2: E-commerce Shopping Cart

Problem

An e-commerce system needs to manage a customer's shopping cart. The cart contains products and their quantities. Customers can add products, remove products, change quantities, and calculate the total price.

A cart cannot contain a product with a non-positive quantity.

Attribute

- customer ID
- products
- quantities
- coupon

Method

- add item
- remove item
- change quantity
- calculate total
- checkout

Rules

- quantity > 0
- price ≥ 0
- no checkout if cart is empty
- coupon must be valid

A list can store products, but it does not know shopping-cart rules.

Example 2: Designing the ShoppingCart class

Class design

```
CLASS ShoppingCart

    ATTRIBUTE
        customer_id
        items
        coupon

    METHOD
        add_item(product, quantity)
        remove_item(product)
        calculate_total()
        checkout()
```

Method pseudocode

```
ADD_ITEM(product, quantity)
    IF quantity <= 0
        RETURN FALSE
    add product and quantity to items
    RETURN TRUE

CALCULATE_TOTAL()
    total = 0
    FOR each item in items
        total = total + item.price * item.quantity
    RETURN total

CHECKOUT()
    IF items is empty
        RETURN FALSE
    total = CALCULATE_TOTAL()
    process payment(total)
    RETURN TRUE
```

Your turn: Design an Investment Portfolio class

Problem

An investment platform manages portfolios for its customers. A portfolio contains several assets, each with an investment weight.

The system should allow investors to add or remove assets, calculate the total portfolio weight, and estimate the expected portfolio return.

For a valid long-only portfolio:

$$w_i \geq 0, \quad \sum_i w_i = 1.$$

Task 1: Design the class

Identify:

- 1 At least **3 attributes**
- 2 At least **4 methods**
- 3 At least **2 rules/invariants** (Homework)

Task 2: Write pseudocode

Write pseudocode for:

- 1 `ADD_ASSET(asset, weight)`
- 2 `TOTAL_WEIGHT()`
- 3 `EXPECTED_RETURN(returns)`

Do not write Python code yet.
Design the object and its algorithms first.

Possible design: Portfolio

One possible class design

CLASS Portfolio

ATTRIBUTE

owner
assets
weights

OPERATIONS

add_asset(asset, weight)
remove_asset(asset)
total_weight()
expected_return(returns)
is_valid()

Next question: how should we represent weights and returns?

Possible pseudocode

```
EXPECTED_RETURN(returns)
  result <- 0
  FOR i <- 0 TO number_of_assets - 1
    result <- result
      + weights[i] * returns[i]

  RETURN result

IS_VALID()
  FOR each weight in weights
    IF weight < 0
      RETURN FALSE
  IF TOTAL_WEIGHT() != 1
    RETURN FALSE

  RETURN TRUE
```

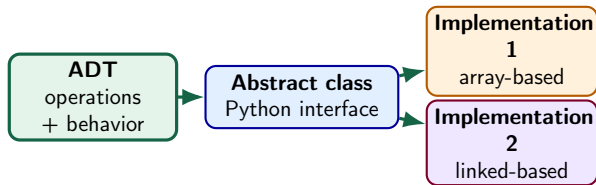
SECTION 3 _____

Inheritance and Abstract Classes for ADTs _____

From ADT specification to Python classes

Recall: an ADT describes behavior

An **Abstract Data Type (ADT)** specifies what kind of data is represented, which operations are available, and what each operation is expected to do. It does **not** require a particular internal representation.



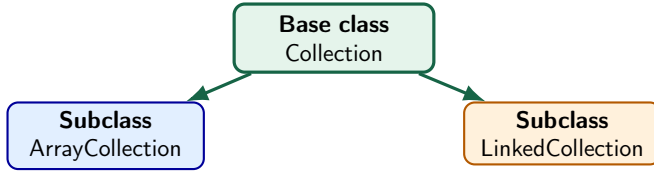
Key idea

An abstract class can express part of an ADT's **common interface**, while subclasses provide concrete implementations.

Inheritance: building a class from another class

Inheritance

Inheritance allows a new class to reuse and specialize attributes or methods defined by another class.



Common behavior

The base class may define operations that every implementation should support.

Different representation

Each subclass may store and manipulate the data differently.

Abstract classes in Python

```
1 from abc import ABC, abstractmethod
2
3 class Collection(ABC):
4
5     @abstractmethod
6     def add(self, value):
7         pass
8
9     @abstractmethod
10    def remove(self, value):
11        pass
12
13    @abstractmethod
14    def __len__(self):
15        pass
```

ABC

ABC means **Abstract Base Class**.

@abstractmethod

Marks a method that concrete subclasses are expected to implement.

Important

An abstract class with unimplemented abstract methods cannot normally be instantiated directly.

The abstract class says what operations are required, not how they are implemented.

A concrete implementation of an abstract class

```
1 class ListCollection(Collection):
2
3     def __init__(self):
4         self._data = []
5
6     def add(self, value):
7         self._data.append(value)
8
9     def remove(self, value):
10        self._data.remove(value)
11
12    def __len__(self):
13        return len(self._data)
```

Inheritance

ListCollection inherits from Collection.

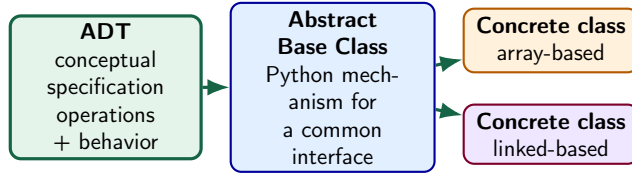
Implementation

The subclass chooses a Python list as its internal representation.

Contract

It provides implementations for all required abstract methods.

ADT, abstract class, and implementation



Keep the concepts separate

- An **ADT** is a conceptual specification.
- An **abstract class** is a programming-language mechanism.
- A **concrete class** implements the operations using a particular representation.

One ADT may have many different concrete implementations.

Example: representing a Stack ADT

Abstract interface

```
1 from abc import ABC, abstractmethod
2
3 class Stack(ABC):
4     @abstractmethod
5     def push(self, value):
6         pass
7
8     @abstractmethod
9     def pop(self):
10        pass
11
12    @abstractmethod
13    def top(self):
14        pass
15
16    @abstractmethod
17    def is_empty(self):
18        pass
```

One concrete implementation

```
1 class ArrayStack(Stack):
2     def __init__(self):
3         self._data = []
4
5     def push(self, value):
6         self._data.append(value)
7
8     def pop(self):
9         return self._data.pop()
10
11    def top(self):
12        return self._data[-1]
13
14    def is_empty(self):
15        return len(self._data) == 0
```

Important

The Stack ADT defines the required behavior. ArrayStack is one possible implementation; another implementation could use linked nodes.

Quick check: interface or implementation?

For a Stack, classify each statement as part of the **ADT specification** or an **implementation decision**.

- ① The most recently inserted element is removed first.
- ② The operation `push(x)` inserts an element.
- ③ Elements are stored inside a Python list.
- ④ When the internal array is full, its capacity is doubled.
- ⑤ `top()` returns the next element to be removed without removing it.

Central question

Which statements describe **what a Stack does**, and which describe **how one particular Stack is implemented**?

Quiz: ADT, abstract class, or implementation?

For each statement, decide whether it is mainly about an **ADT**, an **abstract class**, or a **concrete implementation**.

- ① A stack removes the most recently inserted element first.
- ② `class Stack(ABC)` declares required operations in Python.
- ③ `ArrayStack` stores elements inside a Python list.
- ④ `push(x)`, `pop()`, and `top()` are required stack operations.

SECTION 4

Vector as a Useful Abstraction

We already have a list. Why create a Vector class?

```
1 u = [5, -2, 3]
2 v = [1, 4, 2]
3
4 print(u + v)
```

Python list result

$[5, -2, 3, 1, 4, 2]$

What mathematics expects

$$\langle 5, -2, 3 \rangle + \langle 1, 4, 2 \rangle = \langle 6, 2, 5 \rangle$$

The abstraction problem

A Python list can **store** coordinates, but its operations do not represent vector mathematics.

A Vector may use a list internally, but a Vector is not a list.

Design the Vector before coding

Attribute

- coordinates
- dimension (derived from the coordinates)

Methods

- dimension / length
- get coordinate j
- set coordinate j
- add another vector
- compare vectors

Desired method

$$u = \langle 5, -2, 3 \rangle, \quad v = \langle 1, 4, 2 \rangle$$

$$u + v = \langle 6, 2, 5 \rangle$$

$$u[1] = -2, \quad \text{len}(u) = 3$$

Quick check

Which of these method are already provided by a Python list, and which are not?

Pseudocode for vector addition

```
1 METHOD ADD
2 INPUT: two Vector objects, self and other
3 OUTPUT: a new Vector containing their sum
4
5 IF dimension(self) != dimension(other) THEN
6     report an error
7
8 result = new Vector(dimension(self))
9
10 FOR j = 0 TO dimension(self)-1 DO
11     result[j] = self[j] + other[j]
12
13 RETURN result
```

Design decisions

- Check compatible dimensions.
- Do not modify either input vector.
- Return a new Vector.

Again

The algorithm is defined before the Python implementation.

A first Vector class

```
1 class Vector:
2     def __init__(self, d):
3         self._coords = [0] * d
4
5     def __len__(self):
6         return len(self._coords)
7
8     def __getitem__(self, j):
9         return self._coords[j]
10
11     def __setitem__(self, j, value):
12         self._coords[j] = value
```

Python expression → method

Vector(3)	__init__
len(v)	__len__
v[2]	__getitem__
v[2]=5	__setitem__

Make + mean vector addition

```
1 class Vector:
2     # __init__, __len__, __getitem__, and
3     # __setitem__ are defined as before.
4
5     def __add__(self, other):
6         if len(self) != len(other):
7             raise ValueError(
8                 "dimensions must agree")
9
10        result = Vector(len(self))
11
12        for j in range(len(self)):
13            result[j] = self[j] + other[j]
14
15        return result
```

Now

$$u + v = \langle 6, 2, 5 \rangle$$

Operator overloading

The special method `__add__` defines what the operator `+` means for `Vector` objects.

Why vectors matter in finance and machine learning

Finance

Portfolio weights can be represented as

$$w = [w_1, w_2, \dots, w_n].$$

Asset returns can be represented as

$$r = [r_1, r_2, \dots, r_n].$$

Portfolio return uses their dot product:

$$R_p = w^T r.$$

A useful abstraction connects programming operations to the meaning of the data.

Machine Learning

A customer can be represented by a feature vector such as

$$x = [\text{age}, \text{income}, \text{transactions}, \text{credit score}].$$

A product can also be represented by a feature vector for recommendation or ranking.

Quiz: Vector abstraction

Let

$$u = \langle 2, -1, 4 \rangle, \quad v = \langle 3, 5, 1 \rangle.$$

- ① What should $u + v$ mean for a mathematical Vector object?
- ② Why is Python list addition not enough for this abstraction?
- ③ Which special method can define the behavior of $+$ for a class?

SECTION 5

From Class to Data Structure: Dynamic Array

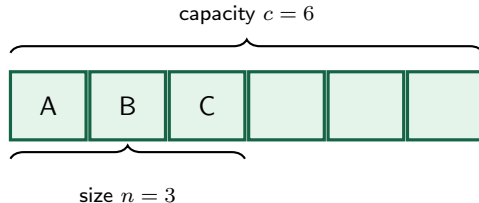
Dynamic Array: a class that implements a data structure

Attribute

- internal array
- current size n
- current capacity c
- invariant: $0 \leq n \leq c$

Public methods

- `len(A)`
- `A[i]`
- `append(value)`



Quick check

Why do we need both *size* and *capacity*?

Pseudocode for append

```
1 METHOD APPEND
2 INPUT: a value to append
3 OUTPUT: the updated dynamic array
4
5 IF size = capacity THEN
6     RESIZE(2 * capacity)
7
8 array[size] = value
9 size = size + 1
```

Key idea

Most appends place the value in an available cell and take $O(1)^a$ time. When the array is full, resizing copies n elements and takes $O(n)$.

^a $O(1)$: constant time, independent of n ; $O(n)$: linear time, proportional to n ; amortized $O(1)$: average constant cost per operation over a sequence.

Why this belongs in a class

Doubling capacity spreads occasional resizing costs across many cheap appends, giving amortized $O(1)$ append time.

Dynamic Array implementation, part 1: public operations

```
1 import ctypes
2
3 class DynamicArray:
4     def __init__(self):
5         self._n = 0
6         self._capacity = 1
7         self._A = self._make_array(self._capacity)
8
9     def __len__(self):
10         return self._n
11
12     def __getitem__(self, k):
13         if not 0 <= k < self._n:
14             raise IndexError("invalid index")
15         return self._A[k]
16
17     def append(self, obj):
18         if self._n == self._capacity:
19             self._resize(2 * self._capacity)
20         self._A[self._n] = obj
21         self._n += 1
```

Public view

`len(A)`, `A[k]`, and `append(obj)`

Hidden details

`_capacity`, `_A`, and the resizing strategy are implementation details.

Dynamic Array implementation, part 2: internal helpers

```
1 def _resize(self, new_capacity):
2     B = self._make_array(new_capacity)
3
4     for k in range(self._n):
5         B[k] = self._A[k]
6
7     self._A = B
8     self._capacity = new_capacity
9
10
11 def _make_array(self, capacity):
12     return (capacity * ctypes.py_object)()
```

What the user sees

Only a simple `append()` operation.

What the class handles

Memory allocation, copying existing elements, and updating capacity.

Abstraction hides complexity behind a simple public operation.

Quiz: Dynamic Array

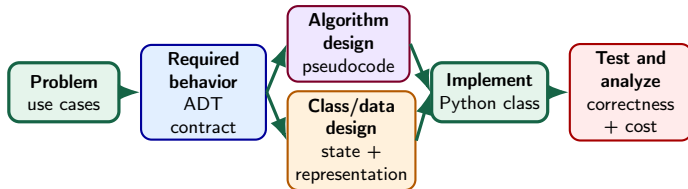
Suppose a dynamic array has current size $n = 4$ and capacity $c = 4$. It uses the doubling rule when the array is full.

- ① What happens before appending the next element?
- ② What are the size and capacity immediately after the append?
- ③ Why can append still be described as amortized $O(1)$?

SECTION 6

Summary

A more precise design workflow



Algorithm and class/data design proceed in parallel and refine each other.

ADT: the behavioral contract

An ADT specifies **what** values are represented, which operations are available, and what rules those operations must follow. It does not prescribe the internal representation.

Pseudocode: the algorithmic steps

Pseudocode describes **how** an algorithm or operation proceeds. It may use ADT operations abstractly, or describe how a concrete method implements one of them.

Key takeaways

- ① Start from the problem and specify the required method before coding.
- ② Good pseudocode is clear, unambiguous, and easy to translate into code.
- ③ AI can assist implementation when the specification is precise, but generated code still requires review and testing.
- ④ A class combines related attribute, method, and rules into one abstraction.
- ⑤ Objects are independent instances of a class; `self` refers to the current instance.
- ⑥ A built-in container may store the attribute but may not provide the method we need.
- ⑦ Data structures such as a Dynamic Array can be naturally implemented as classes.

Central lesson

Specify the method; design the algorithms and representation; then implement, test, analyze, and refine.

After-class practice

- ① Write pseudocode for one business problem: loan approval, shopping-cart total, or portfolio return.
- ② Identify the attribute, methods, and rules of a suitable class for that problem.
- ③ Ask an AI tool to generate a Python implementation from your pseudocode and class specification.
- ④ Compare the generated code with your design and identify any AI-made assumptions.
- ⑤ Extend the `Vector` class with one method such as subtraction or scalar multiplication.
- ⑥ Trace what happens to `size` and `capacity` while appending 10 elements to a `Dynamic Array`.

Final quick check

Match each design question with the most appropriate concept.

- ① What operations should be available?
- ② How should the operation proceed step by step?
- ③ What state and behavior belong together?
- ④ Which methods must subclasses implement?
- ⑤ How are data stored internally?

Concepts

- A. Abstract class
- B. ADT specification
- C. Class design
- D. Concrete implementation
- E. Pseudocode

Write one letter (A–E) for each question.

Any Questions?